

Prompt de lancement — Portage de NOALYSS en Crystal

Table des matières

État des prérequis au 26 septembre 2026	2
Avant le lancement	3
Prompt	3

Usage. Ouvrir une nouvelle session Claude Code dans le dossier de groupe `/Users/philippe/prod-crystal/partiduo`, en mode auto, et coller le contenu du bloc Prompt ci-dessous. Pour reprendre après une interruption, recoller le même prompt : l'équipe repart de `ETAT-PORTAGE.adoc`.

État des prérequis au 26 septembre 2026

Prérequis	État	Constat
Crystal / Shards	cheOK	Crystal 1.19.1, Shards 0.20.0.
Marten ~> 0.7 avec decimal	cheOK	prod-crystal/marten en v0.7.0-20-gdaabb7c2, Marten::DB::Field::Decimal présent.
jose avec RS256	cheOK	v0.2.0, RS256 présent, 102 specs vertes.
saml	cheOK	v1.18.1.3 — intégration seulement.
i18n, pdf-a, pdf-signature	cheOK	Présents dans prod-crystal/.
Modèle de déploiement	cheOK	prod-quimeo/capistrano-quimeo présent.
webauthn	cheOK	v0.1.0 complet : CBOR, COSE (ES256 + RS256), authenticatorData, clientData, et les deux cérémonies verify_registration / verify_authentication. 97 specs vertes contre le jose publié. Chaque contrôle critique — origine, présence et vérification utilisateur, empreinte du RP ID, compteur de signature, vérification de signature — a été éprouvé par mutation. Reste hors périmètre : attestation au-delà de none, et aucun test contre un authentificateur réel.
TOTP (RFC 6238)	cheOK	prod-crystal/totp v0.1.0 : HOTP, TOTP en SHA-1/256/512, base32, URI otpauth://, génération de secrets. 82 specs vertes sur les vecteurs des RFC 4226, 6238 et 4648. Dépôt local, pas encore publié sur GitHub.
Règles de mot de passe CNIL	cheOK	prod-crystal/password-policy v0.1.0 : paliers d'entropie de la délibération CNIL 2022-100, trois pré-réglages, motifs en clés i18n. 36 specs vertes. Dépôt local, pas encore publié.
PostgreSQL local	cheOK	PostgreSQL 18.6, serveur démarré, socket Unix / tmp/.s.PGSQL.5432 autorisée dans prod-crystal/partiduo/.claude/settings.local.json. TCP (127.0.0.1) reste bloqué par le bac à sable.
Sources amont	cheOK	prod-github/noalyss/ : cœur et 10 projets d'extension clonés (dolibarr-noalyss écarté).

Prérequis	État	Constat
Contrat d’extension, facturation électronique	chéRédigés	ADR-003 et ADR-004. Le cœur doit prévoir dès le lot 0 le registre d’extensions, les événements et les champs de facturation française.
Squelette <code>partiduo-app</code> , CI	timèÀfaire	Normal : c’est le lot P, pris en charge par le prompt.
Statut des ADR	chéAkceptés	ADR-001 à ADR-006, le 26 septembre 2026. Maquette de l’interface validée.

Avant le lancement

1. *Fait* — PostgreSQL 18 démarré (`brew services start postgresql@18`) et sa socket Unix autorisée dans `/Users/philippe/prod-crystal/partiduo/.claude/settings.local.json` :

```
{
  "sandbox": {
    "network": {
      "allowUnixSockets": ["/tmp/.s.PGSQL.5432", "/private/tmp/.s.PGSQL.5432"]
    }
  }
}
```

json

2. *Fait* — ADR-001 à ADR-006 au statut « Accepté », maquette validée.
3. *Fait* — questions ouvertes des ADR tranchées dans les ADR ; section « Arbitrages » du prompt réduite aux points absents des ADR.
4. Facultatif : lever la limite de taille des workflows (« Dynamic workflow size » dans `/config`, depuis un terminal `claude interactif`).

Tous les prérequis logiciels sont livrés.

Prompt

```
ultracode – Utilisez un workflow multi-agents pour porter NOALYSS en Crystal. Taille de
↳ workflow libre : lancez autant d'agents que nécessaire.

## Mission

Le projet s'appelle *Partiduo* (portage de NOALYSS, AGPL-3.0-or-later, dépôts
↳ `aloli-crystal/partiduo-*`). Porter le cœur de NOALYSS,
↳ `/Users/philippe/prod-github/noalyss/noalyss-app/` (PHP), en Crystal dans
↳ `/Users/philippe/prod-crystal/partiduo/partiduo-app/`.

La spécification est dans `/Users/philippe/prod-crystal/partiduo/partiduo-docs/` :
↳ `CARTOGRAPHIE-NOALYSS.adoc` et `adr/ADR-001` à `adr/ADR-006`. Je les considère validés. En
↳ particulier :
  - ADR-006 (modules activables) : `partiduo-app` = un *socle* toujours actif + des *modules*
↳ activables par instance – Comptabilité, Facturation, Analytique, Stock. Aucun module
↳ n'appelle un autre module : ils communiquent par le socle et les événements. La CI exécute
↳ les specs dans trois configurations (Comptabilité seule, Facturation seule, les deux). Le
```

markdown

- ↳ module Facturation suit ADR-006 D5 (numérotation **attribuée à la validation, sans trou,**
- ↳ facture intangible, avoir de type 381, mentions obligatoires générées, PDF/A-3 Factur-X).
 - ADR-003 (contrat d'extension) : le cœur **doit accueillir les extensions dès le lot 0 –**
- ↳ registre et manifeste, routes ``/ext/<code>/'`, permissions déclaratives, événements, API
- ↳ publique documentée. **Aucune extension n'est portée dans cette exécution, mais créez**
- ↳ ``partiduo-skel`` (ADR-003 D8) pour prouver que le contrat fonctionne.
- ADR-005 (interface interchangeable) : ``partiduo-app`` ne contient ***aucun*** handler, gabarit
- ↳ ni fichier statique ; il expose le contrat ``Partiduo::Api`` (commandes, requêtes, **objets de**
- ↳ **vue**). L'interface est un dépôt **séparé, `prod-crystal/partiduo/partiduo-ui-bulma`, qui ne**
- ↳ **dépend que de `Partiduo::Api`.** Mettez en place dès le lot P le garde-fou CI de l'ADR-005 D3.
- ↳ L'apparence et l'ergonomie suivent la maquette de référence :
- ↳ ``partiduo-docs/maquette/index.html``.
- ADR-004 D5 : les champs de facturation française (SIREN, SIRET, identifiant de routage,
- ↳ catégorie **d'opération, TVA sur les débits, adresse de livraison**) et l'événement
- ↳ ``payment.matched`` appartiennent au cœur. **Prévoyez-les dans les lots 1 et 2.**

Les extensions amont (``prod-github/noalyss/<projet>``) peuvent être lues pour comprendre ce que le cœur **doit leur exposer.** Toute la documentation produite est en AsciiDoc (``.adoc``), en français.

Autonomie totale

- Ne me posez aucune question. Pour toute ambiguïté, **choisissez l'option la plus simple et**
- ↳ **réversible, consignez-la dans `partiduo-app/DECISIONS.adoc`** (contexte, choix, alternative
- ↳ **écartée), et continuez.**
- Ne vous arrêtez **pas sur un blocage : contournez-le, notez-le dans**
- ↳ ``partiduo-app/BLOCAGES.adoc``, passez au lot suivant.
- Autorisé : **créer `partiduo-app/`, `partiduo-ui-bulma/`, `partiduo-skel/` et**
- ↳ **``partiduo-migrate/`` dans ``prod-crystal/partiduo/``, ``git init`` avec une branche ``development``**
- ↳ (si le bac à **sable refuse l'écriture de ``.git/config``, faites le ``git init`` dans ``${TMPDIR}``**
- ↳ **puis déplacez le dépôt), commiter (commits conventionnels, par lot, sans ligne**
- ↳ **Co-Authored-By), installer des shards, créer des bases PostgreSQL locales préfixées**
- ↳ **``partiduo_`` (dev/test uniquement), corriger un bug avéré **dans les shards****
- ↳ **``prod-crystal/webauthn`` et ``prod-crystal/totp`` (commit dans leur dépôt, noté dans DECISIONS).**
- Interdit : pousser vers un dépôt **distant, modifier quoi que ce soit dans**
- ↳ **``prod-github/noalyss/``, modifier les ADR (proposez plutôt un amendement dans DECISIONS),**
- ↳ **toucher à **d'autres bases que ``partiduo_*`**, modifier d'autres shards de ``prod-crystal/`` que**
- ↳ **ceux cités.**
- PostgreSQL est joignable uniquement par socket Unix (``${TMP}/s.PGSQL.5432``), pas en TCP :
- ↳ utilisez ``psql`` sans ``.h``, et des URL de connexion de la forme
- ↳ ``postgres:///partiduo_dev?host=/tmp``.
- Si PostgreSQL est injoignable, écrivez **quand même le code et les specs, notez-le dans**
- ↳ **BLOCAGES, et continuez.**

Arbitrages

Les questions ouvertes des ADR sont toutes tranchées ***dans les ADR eux-mêmes*** (sections « Questions ouvertes », **barrées et marquées « tranché »**) : version amont figée, CAPTCHA abandonné **et limitation des tentatives, lot 6 reporté (ADR-001) ; SAML + OIDC pluggable,**

↳ **plan d'adressage, codes de récupération, comptes nominatifs, jetons machine (ADR-002).**

↳ **Lisez-les là. Ne sont repris ici que les points absents des ADR ou à ne pas manquer :**

1. Licence : chaque dépôt ``partiduo-*`` **contient le texte intégral de l'AGPL v3 dans**
- ↳ **``LICENSE``, ``license: AGPL-3.0-or-later`` dans ``shard.yml``, et un README qui cite NOALYSS et**
- ↳ **son auteur ; chaque fichier source commence par l'en-tête `SPDX `# SPDX-License-Identifier:`**
- ↳ **`AGPL-3.0-or-later``.**
2. Factur-X : le module Facturation ***produit* le PDF/A-3 Factur-X (XML CII EN16931 embarqué,**
- ↳ **``prod-crystal/pdf-a``, contrôlé par ``pdf-validate``) – c'est dans le périmètre (jalon F).**

- ↳ ***Hors périmètre*** : l'extension ``partiduo-einvoicing`` (ADR-004), c'est-à-dire la connexion à une Plateforme Agréée, les variantes UBL et CII autonomes, la réception, les statuts et le e-reporting.
- 3. Domaine de développement : instances en ``<dossier>.partiduo.localhost``, ``rp.id`` = ``partiduo.localhost`` (ADR-002 D5). Le domaine est un paramètre de configuration, jamais écrit en dur.
- 4. TOTP : compatible avec n'importe quelle application d'authentification, sans en recommander ni en nommer aucune ; paramètres par défaut de la RFC 6238 uniquement (SHA-1, 6 chiffres, 30 s), QR code et secret base32 affichés. ``TOTP::Authenticator#verify`` renvoie un compteur : le stocker (``last_otp_counter``) et le repasser en ``after:``, sinon un code reste rejouable. Aucun utilisateur n'est jamais repris : ``partiduo-migrate`` ne migre ni les utilisateurs ni leurs secrets TOTP.
- 5. Numérotation des factures (ADR-006 D5) : table de compteurs verrouillée par ``SELECT ... FOR UPDATE``, ***jamais*** une séquence PostgreSQL ; specs d'absence de trou (validation annulée) et de doublon (validations concurrentes).
- 6. Pièces jointes : le ***stockage*** est au socle (ADR-006 D1) ; la boîte « Justificatifs à traiter » est l'extension ``partiduo-document``, hors périmètre.
- 7. TVA : BE et FR, isolées dans ``vat/be/`` et ``vat/fr/``.

Stack imposée

Crystal 1.19.1, Marten ≥ 0.7 (source locale ``/Users/philippe/prod-crystal/marten``, doc FR ``/Users/philippe/prod-crystal/marten-docs-fr``), PostgreSQL 18, HTMX + Bulma + Opal. Chaque écran démarre en HTMX ; Opal fait partie de la stack et prend le relais dès qu'un besoin d'ergonomie est constaté (saisie, lettrage, rapprochement – ADR-001 D4, ADR-005 D5) :
 ↳ consignez chaque passage en Opal dans DECISIONS. Réutilisez en priorité les shards maison de ``/Users/philippe/prod-crystal/`` (``jose``, ``webauthn``, ``totp``, ``password-policy``, ``saml``, ``pdf-a``, ``pdf-validate``, ``pdf-signature``). Pour les traductions, utilisez l'i18n intégré de Marten (règles de pluriel des 24 langues de l'UE, ADR-005 D7), pas le shard maison ``i18n`` ; livrez fr, en et nl avant d'en écrire ou d'en importer un. Montants toujours en ``decimal``, jamais en ``Float``. Déploiement sur le modèle de ``/Users/philippe/prod-quimeo/capistrano-quimeo``.

Lot P – prérequis à terminer en premier

- ``prod-crystal/webauthn``, ``totp``, ``password-policy`` et ``authn`` sont livrés : utilisez-les en dépendance ``path:`` tant qu'ils ne sont pas publiés sur GitHub, sans les réécrire.
- Règles de mot de passe : utilisez ``prod-crystal/password-policy`` (préréglage « 14 caractères sans caractère spécial obligatoire » par défaut, ``satisfies_use_case?`` affirmé dans les specs ; motifs de refus en clés i18n). Limite de 71 octets (BCrypt de Crystal).
- ↳ ``prod-crystal/authn`` v0.3 (ex-``kemal-auth``) fournit hachage BCrypt, jetons JWT, sessions par cookie, invitations et réinitialisation de mot de passe ; ``Authn::Password.hash(policy:)`` lève ``PolicyError``. La validation des champs propres d'un utilisateur (adresse, nom, rôle – dont ``comptable``, ADR-002 D4) est de la responsabilité du cœur, avec des messages en clés i18n.
- Squelette ``partiduo-app`` : projet Marten, applications de l'ADR-001 § Organisation du code
 - ↳ dont ``src/modules/`` : registre ``Partiduo::Modules``, manifeste avec ``depends_on`` et ``depends_on_any``, permissions, événements (ADR-003 D2, ADR-006) –, CI (GitHub Actions : format, ameba, specs avec service PostgreSQL), README.adoc.
 - Squelette ``partiduo-ui-bulma`` : intégration d'Opal en place dès ce lot – compilation sans npm ni Node, JavaScript produit versionné et servi par Marten, un exemple minimal fonctionnel couvert par une spec – pour que le passage d'un écran en Opal ne soit jamais un chantier d'infrastructure (ADR-005 D5).

Ordre et parallélisation

Plan de lots de l'ADR-001 : P → 0 → 1 → (2 et F en parallèle) → M → 3 → 4 → 5 → 6. Partiduo démarre de zéro (aucune donnée à reprendre), mais l'outil de reprise ``partiduo-migrate`` est

↳ obligatoire (ADR-001 D5) : c'est un *dépôt séparé*,

↳ `prod-crystal/partiduo/partiduo-migrate`, qui dépend de `partiduo-app` comme shard et

↳ n'écrit que par ses services publics ; source FEC d'abord, base NOALYSS ensuite.

- Un agent « architecte » pose le squelette et les conventions partagées ; rien d'autre ne démarre avant que le squelette compile.
- Ensuite, parallélisez par application Marten (`core`, `accounting`, `cards`, `vat`, `analytic`, `reporting`...) en respectant les dépendances de modèles.
- Pour chaque lot : un agent implémente, un agent `ruby-crystal-tester` écrit les specs, un agent `ruby-crystal-reviewer` relit ; corrigez ses remarques avant le commit.
- Chaque agent lit le code PHP et le SQL d'origine correspondant

↳ (`prod-github/noalyss/noalyss-app/`) pour reproduire les règles métier, pas seulement les écrans.

Définition de « terminé » pour chaque lot

- `crystal tool format src/ spec/` puis `crystal spec` : 100 % vert.
- `marten migrate` passe sur une base vierge.
- Contraintes d'intégrité comptable en base (écriture déséquilibrée refusée par PostgreSQL), testées.
- Commit du lot.

Jalon minimal exigé à la fin

P + 0 + 1 fonctionnels : `bin/partiduo-provision` crée une instance vierge avec le plan comptable de son régime (FR ou BE), on s'y enrôle, on crée et consulte plan comptable, fiches, exercices et journaux. `partiduo-skel` s'active sur l'instance, affiche sa page sous `/ext/SKEL/` pour un profil autorisé, la refuse aux autres, et reçoit l'événement `entry.posted`. Continuez ensuite aussi loin que possible, en visant le second jalon 2 + M : saisie des écritures, puis `partiduo-migrate` importe un FEC (générez-en un de démonstration, par exemple depuis une base NOALYSS créée avec les scripts SQL de `noalyss-app/`) avec un rapport de réconciliation juste au centime. Troisième jalon, F : sur une instance « Facturation seule » puis « Comptabilité + Facturation », un devis transformé en facture validée (numéro attribué, PDF/A-3 Factur-X valide selon `pdf-validate`), puis payée ; avec la Comptabilité, l'écriture de vente est générée et lettrée.

Reprise

Tenez à jour `partiduo-app/ETAT-PORTAGE.adoc` : lots terminés, lot en cours, prochaines étapes, commandes pour lancer l'application et les tests. Si ce fichier existe déjà au lancement, reprenez là où il s'arrête au lieu de recommencer.

Livrable final

Un résumé : lots terminés, nombre de specs, commandes pour lancer, ce qui reste à faire, et le contenu de `DECISIONS.adoc` et `BLOCAGES.adoc` à relire.